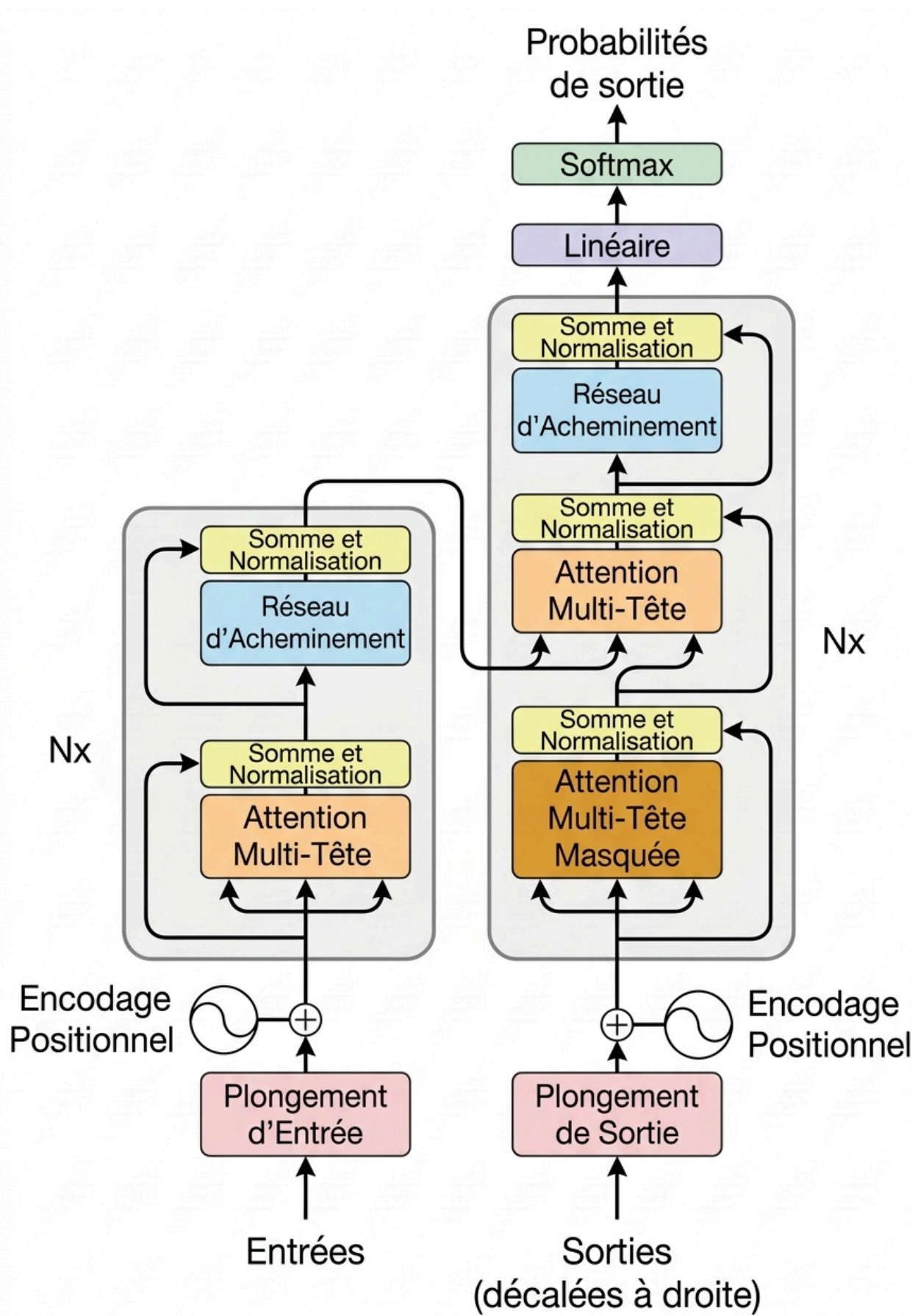




Comprendre l'Architecture Transformer : Le Moteur des IA Modernes

Ce guide pédagogique reprend et explique le schéma canonique de l'architecture **Transformer**, issue de l'article fondamental de Google publié en 2017 : « *Attention Is All You Need* ». Cette architecture est aujourd'hui la colonne vertébrale des modèles de langage comme ChatGPT, Claude, Gemini et bien d'autres.

1. Vue d'ensemble du schéma

Le schéma est divisé en deux colonnes principales :

- **La colonne de gauche (L'Encodeur)** : Reçoit le texte source (par exemple, une phrase en français), l'analyse et en extrait une représentation riche de son contexte et de son sens.
- **La colonne de droite (Le Décodeur)** : Reçoit ce qui a déjà été généré, consulte ce que l'encodeur a compris, et produit le mot suivant (par exemple, la traduction en anglais), un token à la fois.

2. La phase de préparation : Entrées et Encodage

Les ordinateurs et réseaux de neurones ne manipulent pas directement du texte mais des représentations mathématiques (vecteurs de nombres).

Composant	Rôle et Fonctionnement	Exemple Concret
Plongement (Embedding)	Transforme chaque mot ou sous-mot (token) en une suite de nombres de sorte que les mots ayant un sens proche aient des vecteurs	« Chat » et « Félin » se retrouvent situés à proximité dans l'espace vectoriel.

Composant	Rôle et Fonctionnement	Exemple Concret
	géographiquement proches.	
Encodage Positionnel	Contrairement aux anciens modèles qui lisaient séquentiellement, le Transformer traite tous les mots en même temps. On ajoute donc un signal d'ordre à chaque mot.	Permet de distinguer « Le chat mange la souris » de « La souris mange le chat ».

3. Le Bloc Encodeur (Colonne de gauche)

Ce bloc traite et affine l'information d'entrée à travers plusieurs couches :

1. **Attention Multi-Tête (Multi-Head Attention)** : C'est la révolution du modèle. Chaque mot compare sa signification à tous les autres mots de la phrase simultanément pour ajuster sa compréhension selon le contexte. Plusieurs « têtes » travaillent en parallèle, chacune se focalisant sur des aspects différents (grammaire, accords, synonymes).
2. **Réseau d'acheminement (Feed-Forward Network)** : Un réseau de neurones standard appliqué indépendamment à chaque position pour intégrer et transformer les caractéristiques extraites par l'attention.
3. **Somme et Normalisation (Add & Norm)** : Des connexions résiduelles (« raccourcis ») associées à une étape de normalisation qui permettent aux signaux d'apprendre efficacement sans déformation ou disparition du gradient.
4. **Le multiplicateur Nx** : Ce bloc complet est empilé N fois (ex. 6 ou 12 couches consécutives) pour bâtir une compréhension de plus en plus abstraite.

4. Le Bloc Décodeur (Colonne de droite)

Le décodeur est chargé de la génération textuelle :

1. **Attention Multi-Tête Masquée (Masked Multi-Head Attention)** : Le décodeur traite les mots déjà produits. Le masque empêche le modèle de regarder les mots situés dans le futur, garantissant qu'il prédit la suite uniquement d'après le passé.
2. **Attention Multi-Tête Croisée (Cross-Attention)** : Le décodeur prend ses propres représentations et vient interroger les données préparées par l'encodeur. Cela permet au modèle de lier ce qu'il est en train d'écrire au texte source initial.
3. **Réseau d'acheminement & Connexions résiduelles** : Similaires à l'encodeur, répétés N fois.

5. La Production du Résultat (Sortie)

Une fois que le dernier bloc du décodeur a calculé son vecteur final :

- **Couche Linéaire** : Projette le vecteur interne sur l'ensemble du vocabulaire disponible (souvent des dizaines de milliers de mots).
- **Softmax** : Transforme ces scores en pourcentages (probabilités). Le modèle choisit ensuite le mot le plus pertinent (ou échantillonne selon une température donnée), puis réinjecte ce mot pour générer le suivant.

Synthèse pour débutant

En résumé, le Transformer :

- Convertit les mots en coordonnées mathématiques avec leur position.
- Calcule le poids des relations entre tous les mots en même temps grâce à l'**Attention**.
- Utilise un **Encodeur** pour assimiler le contexte et un **Décodeur** pour générer la réponse token par token.